

10



Software Configuration Management

Those who would repeat the past must control the teaching of history.

—Frank Herbert, in *Chapterhouse: Dune*

Change pervades the development process: Requirements change when developers improve their understanding of the application domain, the system design changes with new technology and design goals, the object design changes with the identification of solution objects, and the implementation changes as faults are discovered and repaired. These changes can impact every work product, from the system models to the source code and the documentation. *Software configuration management* is the process of controlling and monitoring change to work products. Once a baseline is defined, software configuration management minimizes the risks associated with changes by defining a formal approval and tracking process for changes.

In this chapter, we describe the following activities:

- *Configuration item identification* is the modeling of the system as a set of evolving components.
- *Promotion management* is the creation of versions for other developers.
- *Release management* is the creation of versions for the client and the users.
- *Branch management* is the management of concurrent development.
- *Variant management* is the management of versions intended to coexist.
- *Change management* is the handling, approval, and tracking of change requests.

We conclude this chapter by discussing project management issues related to software configuration management.

10.1 Introduction: An aircraft example

Passenger aircraft are one of the most complex engineering feats to be attempted by a private corporation. On the one hand, passenger aircraft need to be safe and reliable. On the other hand, they need to be economical. Unlike trains or cars, the aircraft systems cannot be simply shut down in the event of a failure. Unlike NASA, airlines need to post a profit and pay dividends to their stockholders. These requirements result in very complex designs that include many redundancies and diagnostic systems. A Boeing 747, for example, is composed of more than 6 million parts. In order to develop and maintain such complex systems, at the same time remaining economically competitive, aircraft manufacturers extend the life of a model by incrementally improving on the same design. The Boeing 747, for example, was first released in 1967 and is still in production at the time of writing. It has gone through four major revisions, the latest, the 747-400, released in 1988. Another approach to dealing with the high complexity of aircraft is the reuse of as many parts and subsystems across aircraft models as possible. Consider the following example.

Airbus A320

In 1988, Airbus Industries, a consortium of European aircraft industries, released the A320, the first fly-by-wire passenger aircraft. The pilots control the plane like an F16 fighter jet by using a small joystick located on their side. Digital impulses are then transmitted to a computer which interprets and relays them to the wings and tail controls. Unlike hydraulic systems, computerized control allows for envelope protection, which prevents the pilot from exceeding certain parameters, such as maximum and minimum speed, maximum angle of attack, and maximum G-values. The A320 has 150 seats and is targeted for short-to-medium-haul routes.

A321 and A319

The commercial success of the A320 allowed Airbus to work on two derivative airplanes, the A319, a shorter, 124-seat version, and the A321, a longer, 185-seat version. Changing the length of a basic design to obtain a new aircraft has been a standard practice in the industry. This allows the manufacturer to save on cost and time by leveraging off an existing design. It also saves operating costs for airlines, who then only need to maintain one stock of spare parts. In this case, however, Airbus pushed the concept even further, ensuring that all three aircraft have the same cockpit controls and the same handling characteristics. The A321, for example, has slotted flaps and slightly modified wing controls to make the plane feel like the A320, even though the A321 is longer. Consequently, any pilot certified for one of the aircraft can fly the other two. This results in cost savings for airline operators who need to train pilots only once, share flight simulators, spare parts, and maintenance crews for all three versions.

A330 and A340

Pursuing this philosophy even further, Airbus paid great care to the handling characteristics of the A330 and A340. These two aircraft, built for the long haul and ultra long haul markets, can carry twice as many passengers as the A320 up to three times as far. They have the same cockpit layouts and fly-by-wire system. Pilots trained on the A320 family can fly the A330 or the A340 with minimal retraining, further reducing the operating costs of the airline. Comparatively, the handling characteristics of a 737 (comparable to the A319) are very different from a 747 (comparable to the larger version of the A340).

Incremental refinement and subsystem reuse are not without problems. Each change that is made to a single aircraft needs to be carefully evaluated in the context of the other two. For example, if we decide to install a new, more-efficient powerplant on the A320, we would need to evaluate if the same powerplant could also be used on the A319 and the A321, to retain the advantage of sharing parts across all three models. Then, we need to evaluate if the handling of each aircraft changes significantly, in which case we may need to modify the computerized control software to account for this. Otherwise, all pilots flying the A320 would need to be retrained and recertified, without mentioning the loss of a common platform and training program for all three aircraft. Then, the modified aircraft would need to be recertified by the governing authority (e.g., the Federal Aviation Agency in the United States, the Joint Aviation Authorities in the European Community) which would decide on the safety of the aircraft and whether new maintenance or pilot training procedures are necessary. Finally, the state of each individual aircraft needs to be carefully documented, such that the correct parts can be replaced. In summary, any change that would threaten the safety of the aircraft, its efficiency, or the portability of pilots across aircraft need to be identified and rejected. These issues require aircraft manufacturers and operators to follow sophisticated version and change control procedures.

Software system development, although usually not quite as complex as a passenger aircraft, nevertheless suffers from similar problems. Software systems have a long life cycle to allow the recouping of initial investment: For example, many Unix systems still contain code dating as far back as the 1970s. Software systems can exist in many different variants: For example, there are flavors of Unix operating systems running on mainframes as well as home PCs and Macintosh computers. Maintenance changes to existing and running software systems need to be controlled and evaluated carefully in order to ensure a certain level of reliability: For example, the introduction of the Euro currency, a simple change in scale, has had a very visible and substantial impact on many financial and business software systems across the globe. Finally, software systems evolve much more rapidly than aircraft, requiring developers to track change, their assumptions, and their impact.

Configuration management enables developers and aircraft manufacturers to deal with change. The first function of configuration management is the identification of configuration items. Which subsystems are likely to change? Which subsystem interfaces should not change? Each subsystem likely to change is modeled as a configuration item and its state labeled with a version number. The fly-by-wire software of the A320 is a configuration item. The device driver for a serial port is a configuration item for a Unix operating system.

The second function of configuration management is to manage change through a formal process. A change request is first logged, then analyzed, and accepted if consistent with the goals of the project. A change request for an aircraft is a thick report listing all the subsystems and contractors involved in the change. A change request for a simple software system can be just an E-mail requesting a new feature. The change is then approved or rejected, depending on the foreseen impact of the change on the overall system.

Finally, the third function of configuration management is to record sufficient status information on each version of each configuration item and its dependencies. By looking at the maintenance book of an A320 and the version numbers of its subsystems, a maintenance engineer is able to tell which subsystems need to be replaced or upgraded. Looking at the latest release of a serial port device driver, its improvements and the changes since the last release, we are able to determine whether we should upgrade to the new driver or not.

Software configuration management has been traditionally treated as a maintenance topic. The distinction between development and maintenance, however, has become blurred, and often, configuration management is introduced quite early in the process. In this chapter, we focus primarily on the early phases of configuration management and briefly address its use during maintenance. But first, we define more formally the concept of configuration management.

10.2 An overview of configuration management

Software configuration management (later referred to simply as configuration management) is the discipline of managing and controlling change in the evolution of software systems [IEEE Std. 1042-1987]. Configuration management systems automate the identification of versions, their storage and retrieval, and supports status accounting. Configuration management includes the following activities.

- **Identification of configuration items.** Components of the system and its work products and their versions are identified and labeled uniquely. Developers identify configuration items after the *Project Agreement* (Section 11.4.5), once the principal deliverables and components of the system are agreed on. Developers create versions and additional configuration items as the system evolves.
- **Change control.** Changes to the system and releases to the users are controlled to ensure consistency with project goals. Change control can be done by the developers, by management, or by a control board, depending on the level of quality required and the rate of change.
- **Status accounting.** The status of individual components, work products, and change requests are recorded. This allows developers to distinguish versions more easily and track issues related to changes. This also allows management to track project status.
- **Auditing.** Versions selected for releases are validated to ensure the completeness, consistency, and quality of the product. Auditing is accomplished by the quality control team.

In addition, the following activities are often considered part of configuration management [Dart, 1991].

- **Build management.** Most configuration management systems enable the automatic building of the system as developers create new versions of the components. The

configuration management system has sufficient knowledge of the system to minimize the amount of recompilation. It may also be able to combine different versions of components to build different variants of the system (e.g., for different operating system and hardware platforms).

- **Process management.** In addition to change control, projects may have policies about the creation and documentation of versions. One such policy can be that only syntactically correct code can be part of a version. Another policy can be that builds are attempted (and should succeed) every week. Finally, the configuration management process includes policies for notifying relevant developers when new versions are created or when a build fails. Some configuration management systems enable developers to automate such work flows.

Traditionally, configuration management is seen as a management discipline, helping project managers with change control, status accounting, and auditing activities ([Bersoff et al., 1980] and [IEEE Std. 1042-1987]). More recently, however, configuration management has also been seen as a development support discipline, helping developers deal with the complexity associated with large numbers of changes, components, and variants [Babich, 1986]. In this chapter, we focus in more detail on the latter view and only briefly address the activities of change control and status accounting.

In our view, configuration management pervades the software life cycle. It begins with the identification of configuration items after the deliverables and principal components of the system have been defined. It continues throughout development as developers create versions of work products during analysis, system design, object design, and implementation. With rationale maintenance (see Chapter 8, *Rationale Management*), configuration management is the principal tool available to developers for dealing with change.

Next, let us focus in more detail on the concepts of configuration management.

10.3 Configuration management concepts

In this section, we present the main concepts of configuration management (Figure 10-1). As often as possible, we use the same terminology as the IEEE guidelines on configuration management [IEEE Std. 1042-1987]:

- A **configuration item** is a work product or a piece of software that is treated as a single entity for the purpose of configuration management. A composite of configuration items is defined as an **configuration management aggregate**. (called CM aggregate for short). Fly-by-wire software of the A320 is a configuration item. The A320 is a CM aggregate. The serial port device driver is a configuration item. The Linux operating system is a CM aggregate.
- A **change request** is a formal report, issued by a user or a developer, requesting a modification in a configuration item. For example, the *Engineering Change Proposal*

[MIL Std. 480], the standard change request form in the United States government, is seven pages long. An informal change request can be a one-line E-mail message.

- A **version** identifies the state of a configuration item or a configuration at a well-defined point in time. Given a CM aggregate, a consistent set of versions of its configuration items is defined as a **configuration**. A configuration can be thought as a version of a CM aggregate.
- A **promotion** is a version that has been made available to other developers in the project. A **release** is a version that has been made available to the client or the users.
- A **repository** is a library of releases. A **workspace** is a library of promotions.

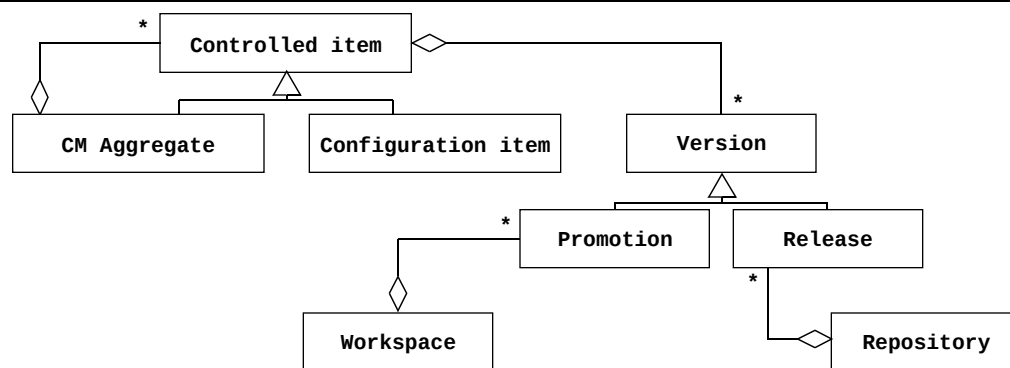


Figure 10-1 Configuration management concepts (UML class diagram).

10.3.1 Configuration items and CM aggregates

A **configuration item** is a work product or a component of a work product that is under configuration management and treated as a single entity for such purposes. For example, the fly-by-wire software of the A320 is a configuration item. During an upgrade, the complete software is replaced. The fly-by-wire software cannot be broken down into smaller components that can be installed independently. Similarly, the device driver for the serial port in any operating system is a configuration item. This component is simple enough that it cannot be broken down further as far as installation is concerned.

A CM aggregate is a composition of configuration items. The 747 is a CM aggregate of six million parts. The Linux¹ operating system is a CM aggregate that includes a process scheduler, a memory manager, many device drivers, network daemons, file systems, and many other subsystems.

1. Linux is a freely available POSIX [POSIX, 1990] operation system created by Linus Torwalds. For more information check <http://www.linux.org/>.

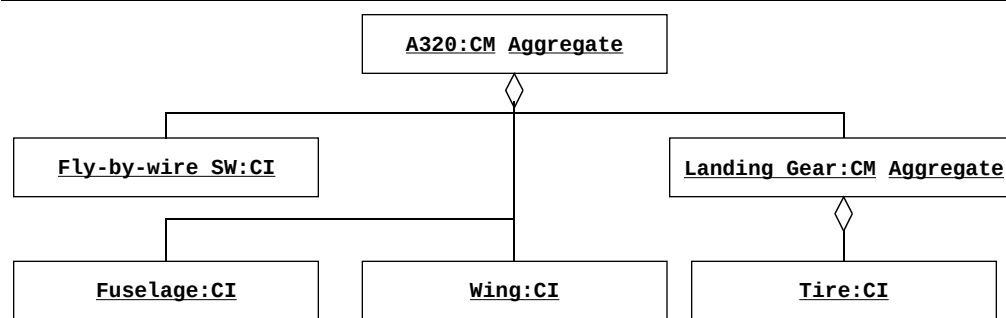


Figure 10-2 An example of CM aggregates and configuration items (UML object diagram).

10.3.2 Versions and configurations

A **version** identifies the state of a configuration item at a well-defined point in time. Successive versions of a work product differ by one or more changes, such as the correction of a fault, the addition of new functionality, or the removal of unnecessary or obsolete functionality. A **configuration** identifies the state of a CM aggregate.

A **baseline** is a version of a configuration item that has been formally reviewed and agreed on, by management or the client, and which can be changed only through a change request. For example, each aircraft needs to go through a rigorous certification process by the governing agency (i.e., the FAA or the JAA) before it can be operated by an airline. Any change to the aircraft after certification requires a formal change process and recertification. This ensures that changes are made consistently with project goals (e.g., safety and reliability) and regulations, and that they are communicated to relevant developers and users (e.g., the airline passengers and the airline operator).

Versions that are intended to coexist are called **variants**. For example (Figure 10-3), the A319, the A320, and the A321 are variants of the same basic aircraft. The principal difference is their length, that is, the number of passenger and freight that they can carry. The A320-200, however, is a version of the A320 that replaced the initial version. In other words, an airline can buy both the A319 and the A320-200, but it cannot buy the older A320-100. In the case of a software system, the system can have a Macintosh variant, a Windows variant, and a Linux variant, each providing identical functionality. A system may also have a standard variant and a professional or deluxe variant, supporting a different range of functionality. Variants share a large amount of code that implements the core functionality, and differences are confined to a small number of lower level subsystems.

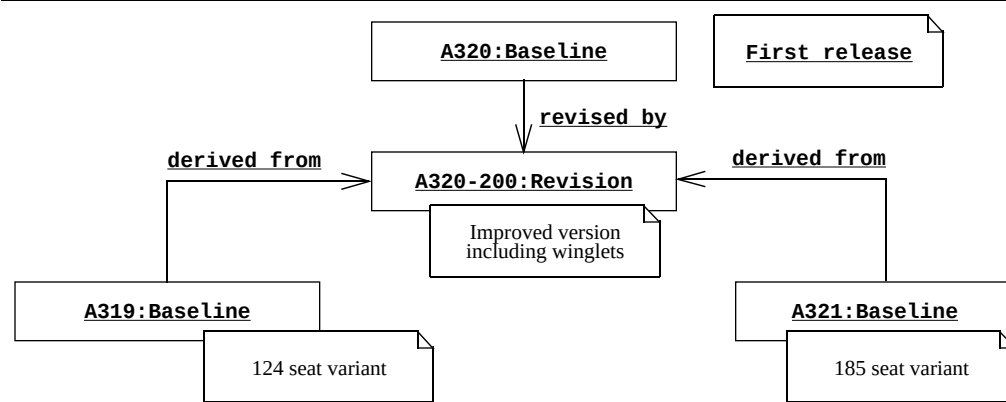


Figure 10-3 Examples of baselines, revisions, and variants (UML object diagram). The A319, A320, and the A321 are all based on the same design. They vary mostly by the length of their fuselage.

10.3.3 Change requests

A **change request** is a formal step initiating the change process. A user, client, or developer discovers a fault in a work product and wants a new feature. The author of the change request specifies the configuration item to which the requests applies to, its version, the problem that needs to be solved, and a proposed solution. In the case of a formal change process, the costs and benefits of the change are then assessed before the change is approved or rejected. In both cases, the rationale of the decision is recorded with the change request.

For example, shortly after the A320 was certified in February 1988, a revised version, the A320-200, was recertified. The new version included a few modifications that yielded a longer range and larger take-off weight. One of these changes was the addition of winglets at the end of the wings that significantly reduced drag and thus reduced fuel consumption. Fuel consumption is a major cost for the airline operator and, thus, a strong selling argument for the airline manufacturer. During the A320 design, a change request was issued, describing the winglet change, its performance evaluation, and its estimated cost. The change was approved, implemented, and the A320-200 passed certification in November 1988. A change request for the Linux operating system kernel is an E-mail message to Linus Torwalds.

10.3.4 Promotions and releases

A **promotion** is a version that is made available to other developers. A promotion denotes a configuration item that has reached a relatively stable state and that can be used or reviewed by other developers. For example, subsystems are promoted for other teams using that subsystem. Subsystems are then promoted for the quality control team to assess their quality. Later, as faults are discovered and repaired, revisions of the subsystem are promoted for reevaluation.

A **release** is a version that is made available to users. A release denotes that a configuration item has met the quality criteria set by the quality control team and can be used or reviewed by the users. For example, the system is released to beta testers for finding additional faults and assessing the perceived quality of the system. As faults are discovered and repaired, revisions of the subsystem are promoted for reevaluation by the quality control team and, when the quality criteria are met, released again to the users.

10.3.5 Repositories and workspaces

Configuration management systems provide a **repository**, in which all configuration items are stored and tracked, and a **workspace**, in which the developer makes changes. Developers create new versions by checking in changes from their workspace to the repository.

A **software library**, as defined in [IEEE Std. 1042-1987], provides facilities to store, label, and identify versions of the configuration items (i.e., documentation, models, and code). A software library also provides functionality to track the status of changes to the configuration items. We distinguish between three types of libraries.

1. The **developer's workspace**, also known as the dynamic library, is used for everyday development by the developers. Change is not restricted and is controlled by the individual developer only.
2. The **master directory**, also known as the controlled library, tracks promotions. Change needs to be approved and versions need to meet certain project criteria (e.g., "Only code that can be compiled without errors may be checked in") before they are made available to the rest of the project.
3. The **software repository**, also known as the static library, tracks releases. Promotions need to meet certain quality control criteria (e.g., "All faults detected by regression tests must be repaired") before a promotion becomes a release.

10.3.6 Version identification schemes

Versions are uniquely identified by developers and systems using a **version identifier**, also called a version number. Some exotic examples include the following.

- The Ada specification went through five successive major versions, named Strawman, Woodenman, Tinman, Ironman, and Steelman [Steelman, 1978].
- The version identification scheme for $T_E X$, a typesetting program for technical texts [Knuth, 1986], is based on decimals of the number π . Each time a bug is found (which is rare) and repaired the version number of $T_E X$ is incremented to add another digit. The current version is 3.14159.

In general, however, version numbers of a given configuration item can be quite large. In this case, developers and configuration management systems use version identification schemes

that support automation more easily, such as sequential numbers with two or three decimals. For example, consider a UML editor, called MUE (for My UML Editor), which is built and released incrementally. We can use a three-digit scheme to distinguish between functional changes, small improvements, and bug fixes (Figure 10-4). The leftmost digit denotes a major version (e.g., overhaul of functionality or of the user interface), the second digit, the minor version (e.g., addition of limited functionality), and the third digit denotes revisions (e.g., corrections). By convention, versions before 1.0.0 denote versions released for the purpose of alpha or beta testing.

Such a simple sequential scheme, however, only works with a sequential series of versions. A **branch** identifies a concurrent development path requiring independent configuration management. Releases are seen by the user as an incremental and sequential development process. The development of different features, however, can be done by different teams concurrently and later merged into a single version. The sequence of versions created by each team is a branch, which is independent from the versions created by the other teams. When versions of different branches need to be reconciled, the versions are **merged**; that is, a new version is created containing selected elements from both ancestor versions.

For releases, the sequential identification scheme is usually sufficient, because the concept of branch is not visible to the users. For developers and configuration management systems, however, this is not sufficient, because branches are often used for supporting concurrent development. The version identification scheme used by CVS [Berliner, 1990], for example, explicitly represents branches and versions. Version numbers include a branch identifier

Three-digit version identification scheme

<version> ::= <configuration item name>.<major>.<minor>.<revision>

<major> ::= <nonnegative integer>

<minor> ::= <nonnegative integer>

<revision> ::= <nonnegative integer>

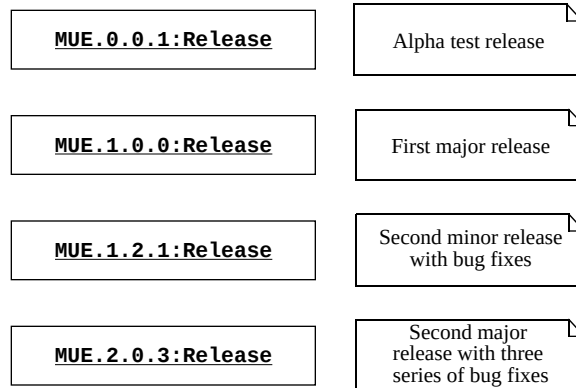


Figure 10-4 Three digit version identification scheme (BNF and UML object diagram).

followed by a revision number. The branch identifier includes the version number from which the branch was started, followed by a unique number identifying the branch. This enables developers to identify to which branch a version belongs and in which order versions of one branch were produced.

In Figure 10-5, two branches are depicted: the main trunk (leftmost package) and the branch 1.2.1 derived from version 1.2 (rightmost package). In the MUE example, the branch may have been derived for the purpose of evaluating competing implementations of the same feature (e.g., support for UML interaction diagrams in MUE). Note that this identification scheme

CVS version identification scheme

<version> ::= <configuration item name>.<version identifier>

<version identifier> ::= <branch>.<revision>

<branch> ::= <version identifier>.<branch number> | <branch number>

<branch number> ::= <nonnegative integer>

<revision> ::= <nonnegative integer>

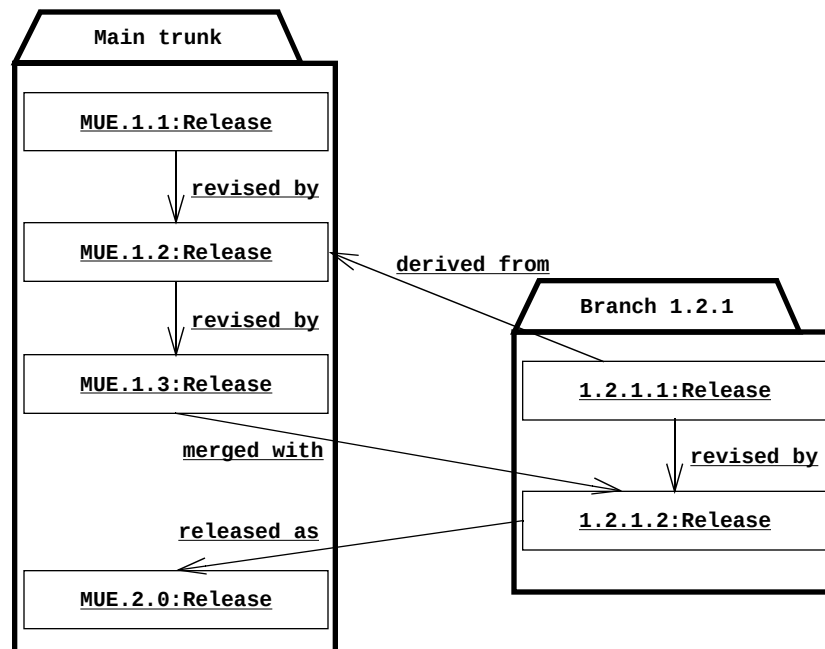


Figure 10-5 CVS version identification scheme (UML object diagram). Branches are identified with the version they were derived from followed by a unique number.

does not identify how versions are merged back onto a branch. In Figure 10-5, version 1.2.1.2 is merged with version 1.3 onto the main trunk to produce version 2.0.

10.3.7 Changes and change sets

The evolution of a configuration item can be modeled in two ways.

- **State-based view.** As a series of versions, that is, as a series of states of the configuration item. Each state is identified by a version number (e.g., the A320-200, MUE.1.0). This is the view most often encountered in practice.
- **Changed-based view.** As a baseline followed by a series of **changes**, also called **deltas**. A change represents the difference between two successive versions, in terms of lines or paragraphs that have been added or removed from the configuration item. Often, repairing a fault or adding functionality to a system requires changes to several configuration items. All the changes to configuration items associated with a single revision of a configuration are grouped into a **change set**. If two change sets do not overlap (i.e., they apply to different and unrelated sets of configuration items), they can be applied to the same baseline in arbitrary order, thus providing more flexibility to the developer when selecting configurations.

Continuing the MUE example, assume that we revised the first baseline twice: MUE.1.1 was released to fix a bug related with classes with no operations, and MUE.1.2 was released to fix a bug related with drawing dashed lines. Each of these revisions corresponded to changes in a single subsystem. The change sets corresponding to each of these revisions is thus independent, and could be applied to the baseline in any order. For example, when the `emptyClassFix:ChangeSet` is applied to the MUE.1.0:Release, we derive the MUE.1.1:Release. Applying the `dashedLineFix:ChangeSet` next results into the MUE.1.2:Release. If, instead, we apply the `dashedLineFix:ChangeSet` first, we would obtain the MUE.1.1a:Release. Applying the `emptyClassFix:ChangeSet` next, however, also results into the MUE.1.2:Release. The change set corresponding to the second baseline, however, depends on both of these revisions. Figure 10-6 illustrates the release history of MUE as a series of change sets and their dependencies.

The change-based view of configuration management is more general than the state-based view. It allows the developer to view related versions of different configuration items as one action. Moreover, when change sets do not overlap, they can be applied to more than one version. This approach is used for delivering bug fixes and minor improvements after a piece of software has been released; each change set is delivered as a separate patch that can be directly applied to the delivered base line or any derived version. As long as patches do not overlap, they can be applied in any order onto the baseline.

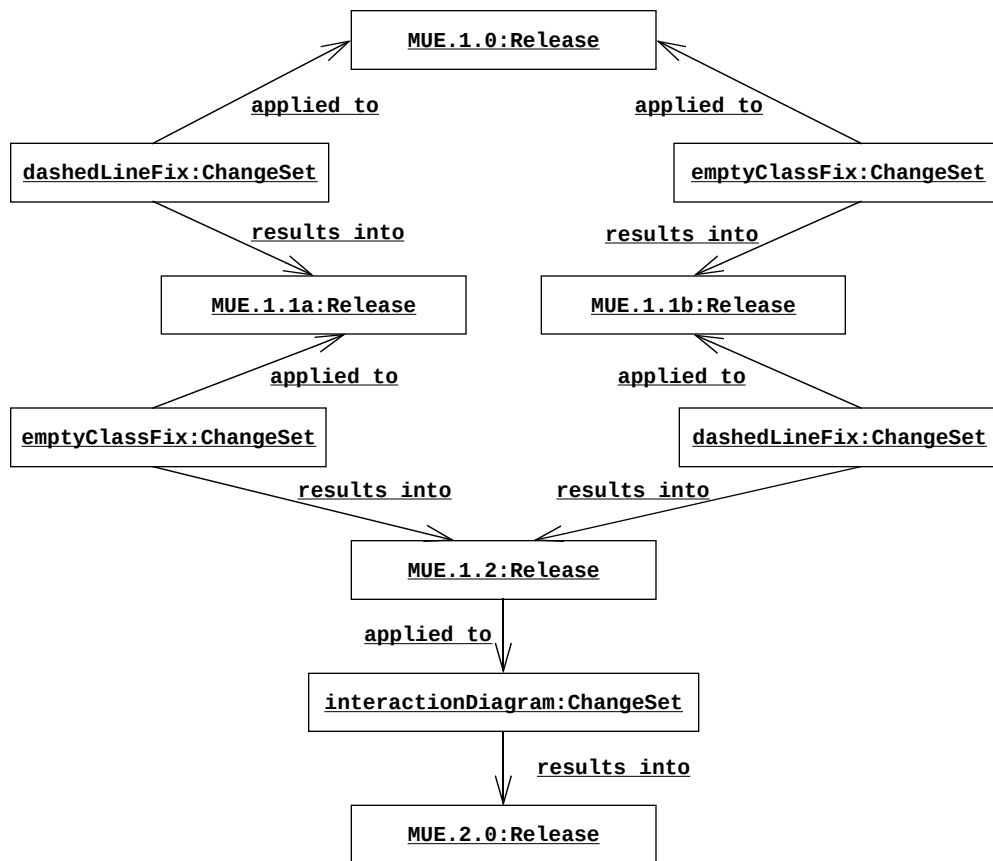


Figure 10-6 Change set representation of the MUE release history (UML object diagram). dashedLineFix:ChangeSet and emptyClassFix:ChangeSet can be applied to the MUE.1.0:Release in arbitrary order because they do not overlap.

10.3.8 Configuration management tools

Due to the importance of configuration management in software development, there are many configuration management and version management tools available to developers. In this section, we briefly describe four of them, RCS [Tichy, 1985], CVS [Berliner, 1990], Perforce [Perforce], and ClearCase [Leblang, 1994].

RCS (Revision Control System), a free tool, controls a repository storing all versions of the configuration items. To obtain a specific version, developers check out a version into their workspace by specifying a version number or a date. To change a configuration item, the developer needs to lock the item first, preventing all other developers from changing the item. When the change is completed, the developer checks the modified item back into the repository,

simultaneously creating a new version and releasing the lock. To optimize storage, RCS only stores the latest version of each configuration item and the differences between each version. The concept of configuration can be realized by attaching a developer specified label to all versions that belong to the configuration. Developers can then check out a consistent set of versions by using the label. Note that this approach does not allow version control of the configuration itself. RCS does not support the concept of branch.

CVS (Concurrent Version System), also a free tool, extends RCS with the concept of branch. Instead of a sequence of differences, CVS stores a tree of differences for each configuration item. CVS also provides tools for merging two branches and detecting overlaps. The change control policy of CVS is also different from RCS. Instead of locking a configuration items, CVS considers each developer to be a separate branch. If only a single developer modified a configuration item between two check-ins, CVS automatically merges the branch onto the main trunk. If CVS detects a concurrent change, it first attempts to merge the two changes and then, in case of overlap, notifies the last developer to check in. With this policy, CVS can support a higher level of concurrent development than RCS.

Perforce is a commercial replacement for CVS. It is based on the same concept of central repository as CVS and RCS. Perforce, however, also supports the concept of change and change set, allowing the developer to more easily track the configuration items that were involved on a given change.

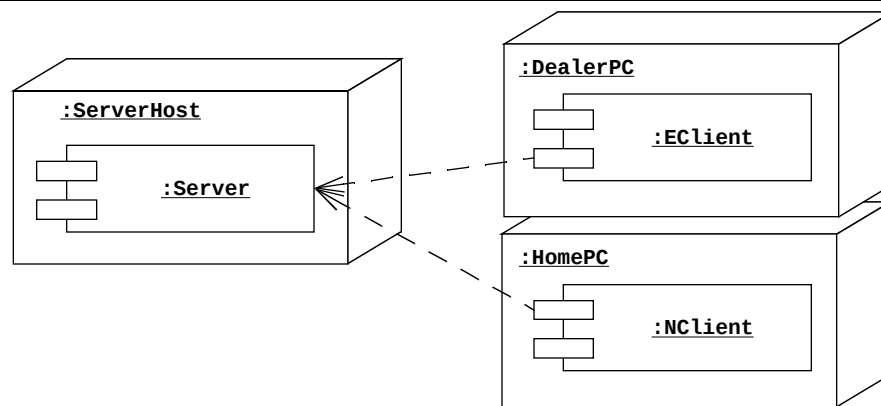
ClearCase, another commercial tool, supports also the concepts of CM aggregates and configurations. A CM aggregate is realized as a directory, which is managed as a configuration item by ClearCase. ClearCase also allows the specification of configurations with rules, selecting versions of each configuration item. A version can be specified with a static rule (i.e., refer to a specific version number) or with a dynamic rule (e.g., refer to the latest version of an item). ClearCase also provides access control mechanisms to define the ownership of each configuration item and configuration.

10.4 Configuration management activities

In the previous section, we described the main concepts of configuration management. In this section, we focus on the activities necessary to define and manage configuration items, promotions, and releases. We also describe activities related with the use of branches and variants for concurrent development. Configuration management activities described in this section include:

- configuration item and CM aggregate identification
- promotion management
- release management
- branch management
- variant management
- change management

For the basis of the examples in this section, we use a distributed car parts catalog called myCarParts. MyCarParts allows car dealers and car owners alike to browse and order parts from their computer. MyCarParts has a client/server architecture, with two types of clients: the `EClient` for expert users, such as car mechanics and part dealers, and the `NClient` for novice users, including car owners who repair their own car. The system requires users to authenticate, which allows it to determine the list price of the parts given the particular user. A user who orders many parts is eligible for volume discounts, whereas an occasional client pays the full list price. The system also tracks the interests of each user, using this information to better optimize the network usage. The user is also sent customized notices about new products or new discount prices. Figure 10-7 displays the subsystem decomposition of myCarParts.

**:DealerPC**

The `:DealerPC` is the machine used by a dealer to order parts. The `:DealerPC` has often a higher bandwidth link to the server.

:EClient

The `:EClient` is located on the `:DealerPC`. It provides functionality to an expert user for finding parts by part identifier, vehicle make and year, and order history. The `:EClient` is designed for high-volume clients, such as car repair shops and part dealers.

:HomePC

The `:HomePC` is the machine used by a car owner to order parts. The `:HomePC` is connected to the server via a modem.

:NClient

The `:NClient` is located on the `:HomePC`. It provides functionality to the novice user for finding parts by description and, in subsequent releases, by clicking on a vehicle map. The `:NClient` is designed for the occasional client, such as a car hobbyist.

:ServerHost

The `:ServerHost` hosts the parts catalog server.

:Server

The `:Server` enables client to retrieve lists of parts by criteria and part entries, to order parts, and to track client activity.

Figure 10-7 MyCarParts subsystem decomposition and hardware allocation (UML deployment diagram).

The development and evolution of the myCarParts system requires the coordinated release of several components.

- The protocol used by the clients and the server to exchange information is occasionally upgraded to support new client functionality and to improve response time and throughput. Because there is no guarantee that all users have the same version of the client, it is necessary for the server to maintain backward compatibility with older clients. Every new revision needs to be tested with older clients to validate this property.
- New client versions may implement functionality that is supported only in new versions of the server. Therefore, the server needs to be upgraded first, before the new client versions can be made available.
- When a new version of either client is available, the user can download a patch. The corresponding hardcopy manual, however, is sent via standard mail. In the event that several versions are released in a short time, a user needs to be able to identify which manual corresponds to which version.

For example, assume that the initial release of myCarParts allows users to browse the catalog using textual information only. Parts have a part identifier, a name, a description, the list of vehicles and years for which this part is manufactured, and cross references to other parts with which it can be assembled. The user can search the database using any of these fields. This can be problematic for novice users who do not know the name of the part they are looking for. In a second release, we address this problem by providing the user with a navigation map of parts. If the user knows what the part looks like and where it is located in the vehicle, he only needs to click on the corresponding part of the map. The change to introduce navigation maps and the releases of the components of myCarParts need to be carefully sequenced, considering the constraints we described before. The navigation map change requires the following sequence of steps.

1. The Server needs to be modified to support the storage and retrieval of subassembly maps.
2. The Server needs to be released and installed.
3. One map per vehicle and year needs to be created and stored in the database.
4. The NClient needs to be modified to use the navigation maps as a possible interface.
5. The NClient needs to be released and installed.

In the following sections, we use myCarParts and the navigation map change as the basis for the examples.

10.4.1 Configuration item and CM aggregate identification

The identification of configuration items and CM aggregates occurs primarily after the project agreement, when a set of deliverables is agreed with the customer, and after system design, when most of the subsystems have been identified. The identification of configuration items and CM aggregates, however, continues throughout the development as the set of deliverables is redefined and as subsystems are added and removed from the subsystem decomposition.

Identifying configuration items and CM aggregates is similar to identifying objects during analysis. It is not algorithmic because some configuration items are trivial to identify (e.g., the RAD, the SDD), while others are more subtle (e.g., a client server protocol definition). Configuration items are self-contained documents or pieces of code whose evolution needs to be tracked and controlled. These include deliverable documents, work products, subsystems, off-the-shelf components that can evolve during the life cycle of the system, and interface descriptions. Usually, most configuration items have already been identified as objects of interest during project planning or system design.

In the myCarParts example, we identify each of the deliverable documents as a configuration item. In the myCarParts *Project Agreement*, the following were defined as deliverables:

- user level documents, including the RAD and the User's Manual (UM);
- system documents, including the SDD and the ODD
- the source code of the system, including its subsystems (depicted in Figure 10-7) and various installation programs

We identify, moreover, the interfaces between subsystems entities whose evolution should be carefully controlled, as changes to these can introduce major problems. We identify two components of the SDD, the *Client Server Protocol Specification* and the *Data Schema Description* as configuration items.

Figure 10-8 depicts the configuration items and the CM aggregates we identified for myCarParts. Because myCarParts subsystems can be released independently, we identify one CM aggregate for each subsystem that includes the configuration items relevant to the subsystem. For example, if we modify the NClient subsystem to include the navigation map functionality, we need to revise the RAD to specify the new use cases, change the NClient UM:CI to explain how this functionality is used, modify the NClient ODD:CI to include new classes for the map, and implement and test the changes in the source code.² The NClient:CMAggregate is then revised to include the most recent versions of the configuration items related to the NClient; before it is released to the users. Note that this change can be done without modifying the

2. The source code, test, and test manual are not depicted in Figure 10-8 for brevity.

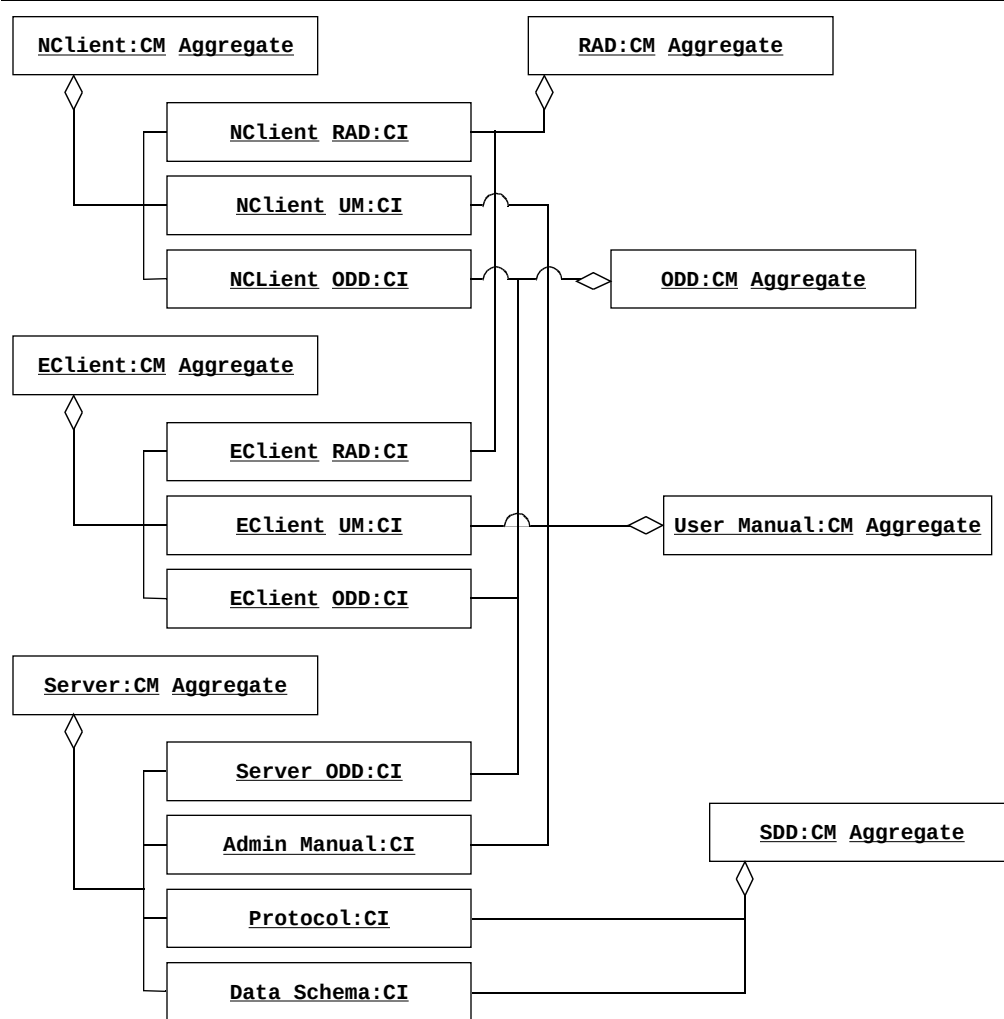


Figure 10-8 Configuration items and CM aggregates for myCarParts (UML object diagram).

EClient:CM Aggregate and, thus, configuration items related to the **EClient** and the **NClient** subsystems are modeled as separate configuration items and CM aggregates.

The three subsystems constituting the myCarParts system, however, are not completely independent. Before we can add navigation maps to the **NClient** subsystem, we need first to provide functionality for storing and retrieving these tables in the **Server** subsystem. Although this functionality is realized and released separately, we still need to make sure that the releases of clients are coordinated with server releases (in the sense that servers need to be upgraded before clients are). For this purpose, we also identify system-level CM aggregates for each of the

deliverables. The RAD:CM Aggregate, the SDD:CM Aggregate, and the ODD:CM Aggregate represent consistent versions of the deliverables. For example, the versions of the ODD:CM Aggregate with the NClient ODD:CIdescribing the navigation map classes also contains the Server ODD:CIdescribing how to store and retrieve these maps.

10.4.2 Promotion management

Creating a new promotion for a configuration item occurs when a developer wants to share the configuration item with others. Developers create promotions to make the configuration item available for review, for the purpose of debugging another configuration item, or for a sanity check. Once the promotion is created and stored in the configuration management system, developers interested in the promotion check it out of the repository. Developers not interested in the promotion continue to work with earlier versions. Once a promotion is created, it cannot be modified. The developer who created the promotion can continue modifying the configuration item without interfering with the developers using the promotion. To distribute new changes, the developer needs to create a new promotion.

For example, consider the navigation map change in myCarParts (Figure 10-9). First, developers modify the analysis model to specify the navigation map use cases and its interaction with the existing use cases, resulting in the NClient RAD.2.0:Promotion. The system design model is then modified to accommodate the storage and downloading of maps, resulting in the Protocol.2.0:Promotion and Data Schema.2.0:Promotion. Then, a first implementation of the server side of the protocol is realized in the Server.2.0:Promotion, which is made available to the novice client team to test their implementation of the navigation map (NClient 2.0:Promotion and above). Several bugs are found in the server while testing the NClient. The server team identifies and addresses these bugs, resulting in the Server.2.1:Promotion and Server.2.2:Promotion. In the meantime, the documentation team revises the NClient UM2.0:Promotion based on the NClient RAD.2.0:Promotion. Note that during this time, the expert client team may be repairing bugs in the EClient.1.5 independently of the navigation map change. To test the EClient, the expert client team continues to use the former release of the server (i.e., Server.1.4). Figure 10-9 illustrates the above scenario by depicting a snapshot of each team's workspace. Even though teams are working toward a consistent system, they may all be working on different promotions of the same components until all components stabilize.

Promotions represent the state of a configuration item at the time it is made available for other developers. A project may usually require that code promotions do not contain compiler errors but make few other constraints to encourage the exchange of work products among teams. Once the quality of configuration items is improved and assessed by the quality control team, the promotion may become eligible for a release.

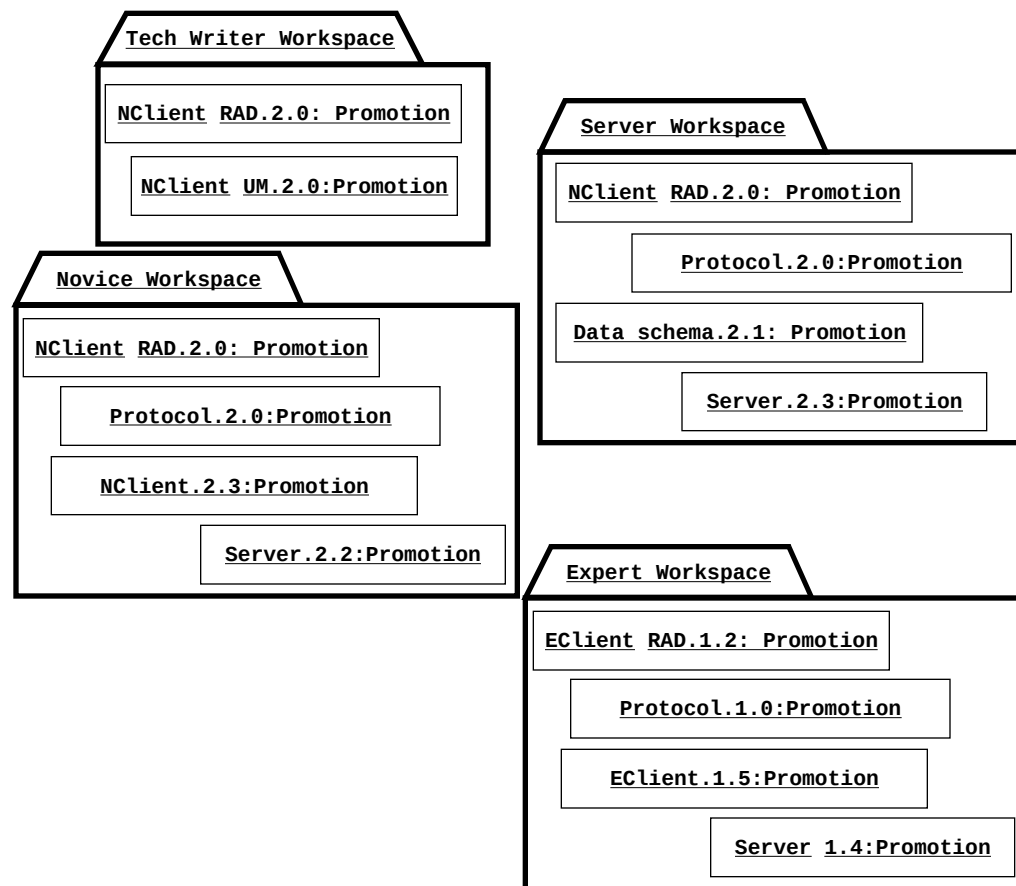


Figure 10-9 Snapshot of the workspaces used by myCarParts developers (UML object diagram). The Novice Workspace, the Tech Writer Workspace, and the Server Workspace contain promotions related to the navigation map functionality. The Expert Workspace, however, contains older and more stable versions. For all configuration items, version numbers of the form 1.x refer to the promotions without navigation maps functionality whereas version numbers of the form 2.x refer to the promotions containing partial or complete implementations of the navigation maps.

10.4.3 Release management

The creation of a new release for a configuration item or an CM aggregate is a management decision, usually based on marketing and quality control advice. A release is made available to offer additional (or revised) functionality or to address critical bugs.

Although creating a release seems similar to creating a promotion, it is a much more complicated and expensive process. Developers deal with change as part of their work. If the new version of a component introduces more problems than it addresses, they can simply roll back to an earlier promotion. A user manual that is not up to date does not interfere with their work given that they know the product. Users, on the other hand, are not in the business of testing software. Discovering inconsistencies in the system and its documentation interferes with their work and can lead them to switch products or contractors. Releases, therefore, are created in a much more controlled process than promotions, attempting to ensure consistency and quality. The quality control team assesses the quality of the individual components of a release and coordinates revisions to defective components. This auditing process allows the quality control team to ensure the consistency of the release with minimal interference with the developer's work.

For example, consider the release of the navigation map functionality for myCarParts (Figure 10-10). We first create a stable promotion of the `NClient.2.4` and the `Server.2.3` that implements the new functionality. The quality control team tests these two promotions against the latest version of the RAD (i.e., `NClient RAD.2.0`). Quality control finds a bug, which is fixed in a subsequent promotion of the `NClient.2.5`. Quality control then tests the current version of the `EClient` (i.e., `EClient.1.5`) with the current version of the server. Satisfied with the state of the software, quality control decides to include `NClient.2.5`, `EClient.1.5` and `Server.2.3` in the next major release of myCarParts. Quality control then checks the user manual (`NClient UM.2.0`) for consistency against `NClient.2.5`. A few additional problems are found and addressed in the promotion `NClient UM.2.1` which is then included in the release under construction. The release is then beta tested with a selected set of users. Two more problems with the `NClient.2.5` and `Server.2.3` are discovered by a beta tester and are addressed in `NClient.2.6` and `Server.2.4`. The software is retested and released to the users as `myCarParts.2.0`.

The administrator of the myCarParts site first upgrades the `Server` and its database schema and observes, during a beta test, if any user discovers unforeseen compatibility problems with the older clients. After the beta test, the administrator then makes available the `NClient.2.6` to the users who would like to use the navigation map functionality. Once the stability of the new release is validated in the field, all users are encouraged to upgrade and older versions of the server and the clients are discontinued. Any change to the documentation or subsystems of myCarParts will be delivered as a patch or as a third release.

The focus of the release process in the myCarParts example is on quality and consistency. The quality control team acts as a gatekeeper between the users and the developers. The assumption behind this process is that users are not interested in debugging the systems they use. They expect that these systems support their work. This assumption does not hold when the users are software developers. In this case, a substantial number of tools used by this group of users are also developed and maintained by them. These can range from simple scripts, making it more convenient to accomplish a repetitive task, to full-fledged programming languages,

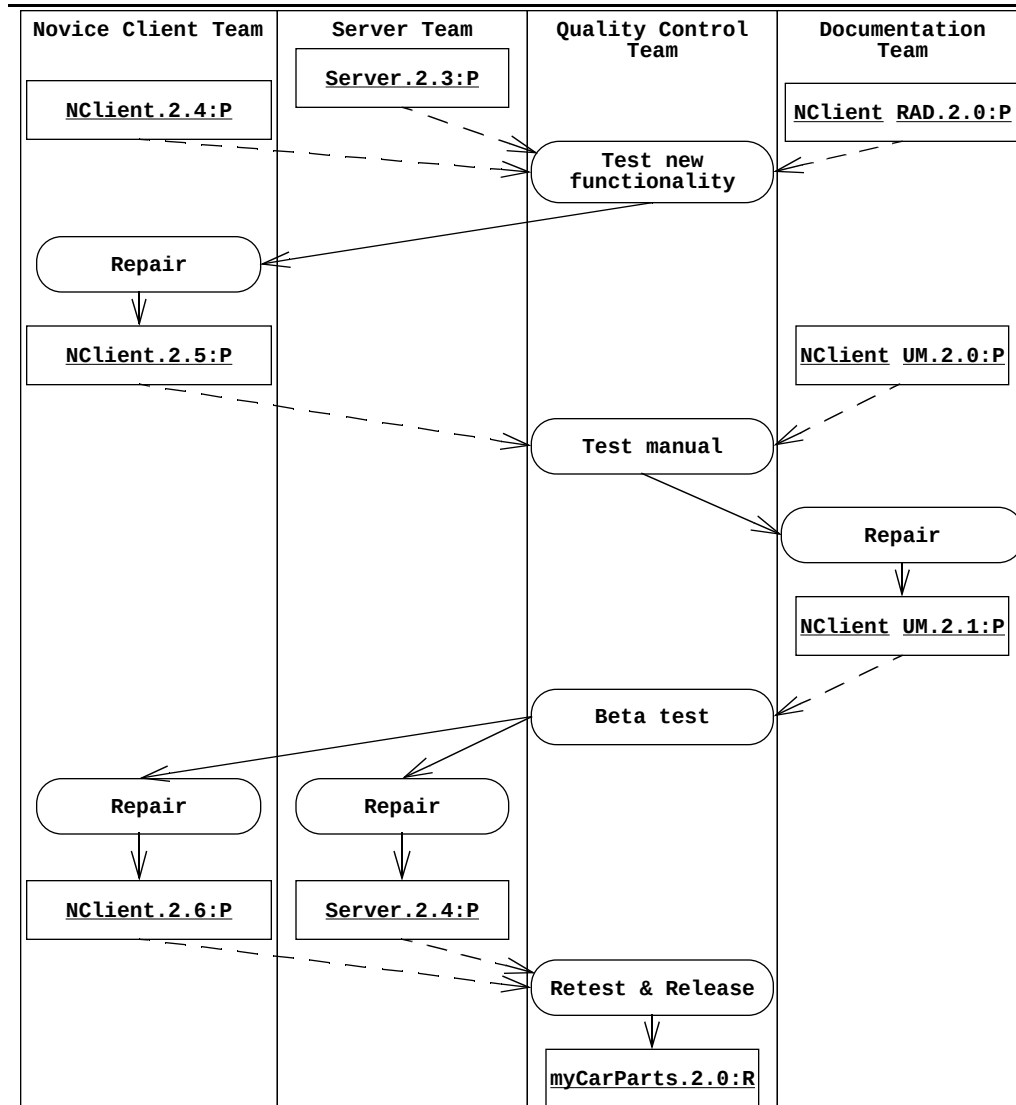


Figure 10-10 Release process for the navigation map functionality of myCarParts.2.0 (UML activity diagram). :P denotes a promotion, :R denotes a release.

syntax editors, or configuration management systems. With the popularity of the Internet among software developers, an increasing number of these ad hoc tools are shared and distributed, especially those that address common problems. This leads to the availability of numerous freely available programs, ranging from simple scripts to operating systems (e.g., Linux). In this situation, users (who are software developers) are willing to test, debug, and send contributed

code to the program author in exchange for early access to the piece of software. In this software development model, known as the bazaar model [Raymond, 1998], releases and promotions become much more similar than in the context of the controlled process we described above.

10.4.4 Branch management

Until now, we have examined the management of promotions and releases in the context of a single change. Various configuration items are revised until the change is completed and then assessed by quality control. During this process, we have dealt with the complexity caused by the difficulty in maintaining consistency among related promotions and in minimizing the introduction of new faults. We have focused, however, only on a single thread of development.

Usually, developers work on multiple improvements concurrently. For example, while developers implement the navigation map functionality in the `NClient` and the `Server`, other developers may be improving the `Server` response time, while yet another group of developers may be extending the `EClient` to store the history of queries issued by the user. When threads of development focus on different subsystems of `myPart` (e.g., navigation maps in the `NClient` and the `Server`, query history in `EClient`), teams can be isolated by working on different configurations of the group same subsystems (e.g., the navigation map effort works with the most recent versions of the `NClient` and the `Server`, while the query history effort works with the most recent version of the `EClient` and an older, stable version of the `Server`). This approach works only if changes impact nonoverlapping sets of components and if subsystems interfaces remain backward compatible. However, when two changes require modifications to the same component, a different approach is required. Concurrent branches and subsequent merging can be used to coordinate changes.

For example, let us focus on two concurrent and overlapping changes: While developers implement the navigation map functionality (described in Section 10.4.2 and in Section 10.4.3), we assign another team the task of improving the response time of the `Server`. Both teams may change the `Server`, the `NClient`, and their interfaces to accomplish their respective goals.

These changes are assigned to different teams because of the difference in their associated risks. The navigation map improvement extends the functionality of `myCarParts` and is well defined. The response time improvement, however, requires experimental work and is open ended: The developers first need to identify performance bottlenecks and design heuristics to speed up common requests. The resulting improvement then needs to be measured and assessed against any loss of reliability and decrease in maintainability. Finally, separating both changes provides us with more flexibility during delivery: If the response time improvement is completed early, we can merge it with the functional improvement and release it at the same time. Otherwise, if the response time improvement takes more time to be completed, we can deliver it later as a patch.

To support both changes concurrently while keeping teams independent, we set up a branch, starting from the latest promotions of the subsystems at the time the changes are approved (see Figure 10-11).

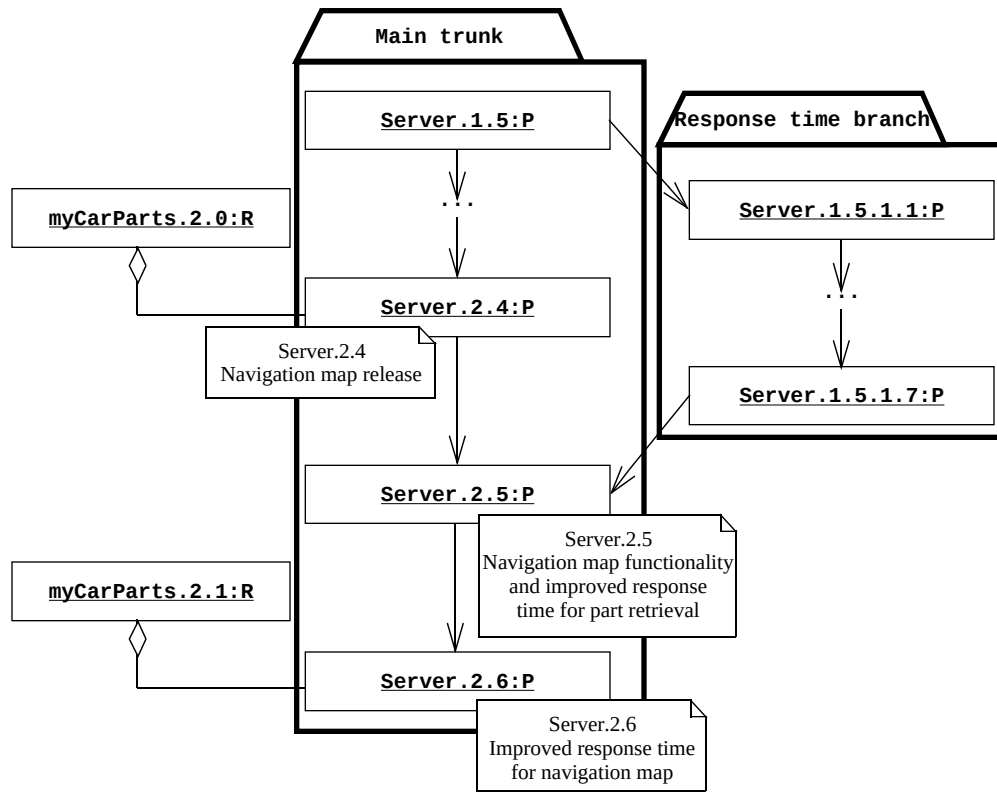


Figure 10-11 An example of branch (UML object diagram, some promotions were omitted for brevity, :P denotes a promotion, :R denotes a release). On the main trunk, developers add the navigation map functionality to myCarParts. On a concurrent branch, developers improve the response time of the server by integrating a cache between the server and the database. The response time improvement is completed after the release of the navigation map functionality and made available as a patch.

The teams working on functional improvements continue working on the main trunk (starting at Server.1.5 and NClient.1.5). The team responsible for the response time improvement works on the branch (starting at Server.1.5.1.1).³ The performance improvement team restricts its changes to the Server subsystem and decides to avoid modifying

3. We use the CVS identification scheme for identifying versions and branches [Berliner, 1990].

the Server interface. Both teams then work independently until they complete their improvements. The navigation map improvement is completed first and made part of the second release of myCarParts, as we saw in Figure 10-10 (as Server.2.4). Soon after, the response time improvement is completed (Server.1.5.1.7 in Figure 10-11). The response time improvement is considered successful, yielding a fourfold decrease in response time for common requests, while marginally extending the client server protocol. At this point, we need to integrate this improvement with the main trunk, hoping to produce a version of myCarParts which has both the navigation map functionality and the fourfold improvement in response time.

Merges can usually be done with the help of the configuration management system, which attempts to merge the most recent parts of the versions to be merged. When conflicts are detected, that is, when both versions contain modifications to the same classes or methods, the configuration management tool reports the problem back to the developer. The developer then resolves the conflict manually. In our example, the tool reports a conflict in the DBInterface class which was modified by both teams (Figure 10-12).

The navigation map team added a method, `processMapRequest()` to retrieve maps from the database. The response time improvement team modified the `processPartRequest()` which retrieves parts from the database given a part id. We construct a merged version of the DBInterface class by selecting the `processMapRequest()` from the main trunk and the `processPartRequest()` from the branch. We then test the revised DBInterface class to make sure we have dealt with all the conflicts, which results in the Server.2.5:PromotionLater, we realize that the same caching mechanism can be used for the `processMapRequest()` which can result in further response time improvements. We modify the `processMapRequest()` method, retest the server, and create the Server.2.6:Promotion

Heuristics for branch management

As we saw in this example, merging two branches is a nontrivial operation. It usually needs to be supported by a configuration management tool and requires substantial intervention and testing by developers. In the myCarPartsexample, illustrating a very simple case, the response time improvement team was careful not to change the Server interface and limited their changes to the Serversubsystem. These constraints minimized the likelihood of an overlap with the changes made by the navigation map team. In general, if no constraints are set, branches can diverge to the point where they cannot be merged. Although there are no reliable ways to address this problem, several heuristics can be used to mitigate the risk of divergent branches.

- **Identify likely overlaps.** Once the development branches are set up, but before design and implementation work starts, developers can anticipate where overlaps could occur. This information is then used to specify constraints for containing these overlaps. Examples of such constraints include not modifying the interface of classes involved in the overlap.
- **Merge frequently.** The configuration management policy can require developers working on a branch to frequently merge with the latest version of the main trunk (e.g., daily,

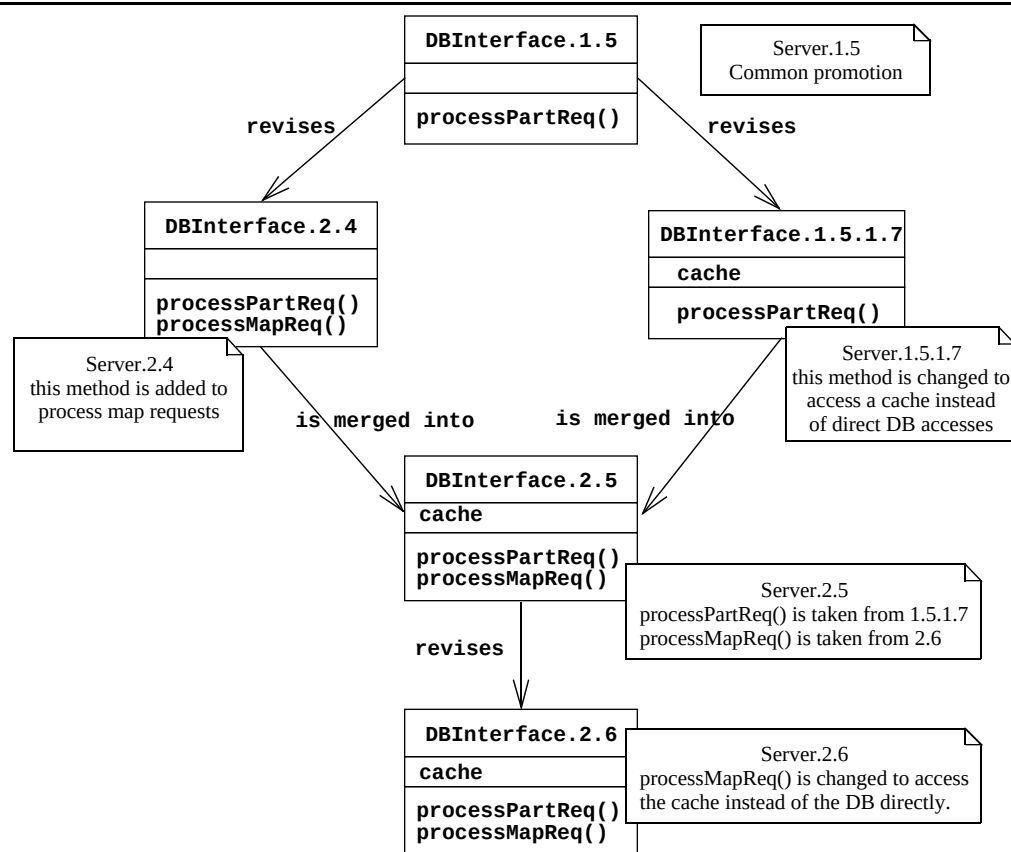


Figure 10-12 An example of merge for the `DBInterface` class of the `myCarParts` system (UML class diagram).

weekly, or whenever a new promotion is created). The merges are only created on the branch and are not propagated back on the main trunk. The policy also may specify that such merges need only to ensure that the code still compiles; that is, the merges need not necessarily address all overlaps. This policy encourages developers to find overlaps early and think about how to address them before the actual merge.

- **Communicate likely conflicts.** Although teams working on different branches need to work independently, they should anticipate conflicts during the future merge and communicate them to the relevant teams. This also has the benefit of improving the design of both changes by taking into account constraints from both teams.
- **Minimize changes on the main trunk.** Minimizing the number of changes to one of the branches to be merged reduces the likelihood of conflicts. Although this constraint is not

always acceptable, it is a good configuration management policy to restrict changes to the main branch to bug fixes and do all other changes on the development branch.

- **Minimize the number of branches.** Configuration management branches are complex mechanisms that should not be abused. Merge work caused by reckless branching may result into substantially more effort than if a single branch was used. Changes that may result in overlaps and conflicts usually depend on each other and can be addressed sequentially. Branches should be used only when concurrent development is required and when conflicts can be reconciled.

In all cases, creating a branch is a significant event in the development. It should require management approval and should be carefully planned.

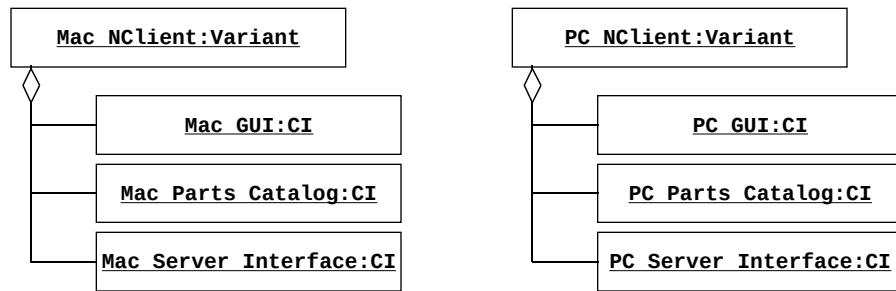
10.4.5 Variant management

Variants are versions that are intended to coexist. A system has multiple variants when it is supported on different operating systems and different hardware platforms. A system also has multiple variants when it is delivered with different levels of functionality (e.g., novice version vs. expert version, standard version vs. deluxe version). Two fundamental approaches are possible when dealing with variants (Figure 10-13).

- **Redundant teams.** A team is assigned to each variant. Each team is given the same requirements and is responsible for the complete design, implementation, and testing of the variant. A small number of configuration items are shared across variants, such as the user manual and the RAD.
- **Single project.** Design a subsystem decomposition that maximizes the amount of code shared across variants. For multiple platforms, confine variant specific code into low-level subsystems. For multiple levels of functionality, confine increments of functionality in individual and mostly independent subsystems.

The redundant-team option leads to multiple smaller projects that share a requirements specification. The single project option leads to a larger, single project with most teams sharing core subsystems. At first sight, the redundant team option leads to redundancies in the project as the core functionality of the system will be designed and implemented multiple times. The single project option seems more efficient, given that a potentially large amount of code can be reused across variants, given a good system design. Surprisingly, the redundant-team option is often chosen [Kemerer, 1997] in commercial developments to avoid organizational complexity.

Redundant team organization



Single project organization

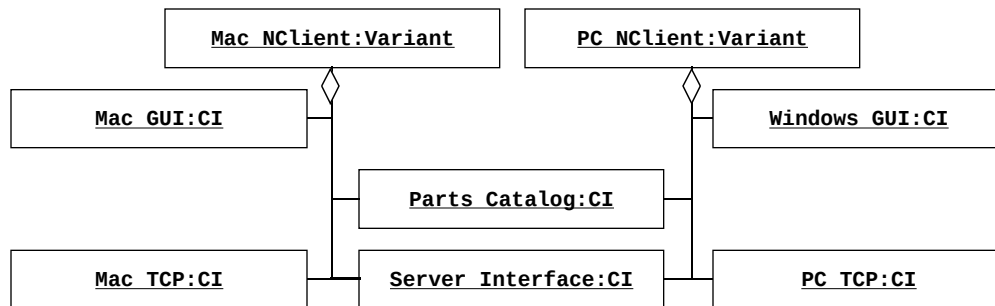


Figure 10-13 Examples of redundant variants and variants sharing configuration items (UML object diagram). In the redundant team organization, the myCarParts NClient's for the Macintosh and the PC are realized independently. In the single project organization, the myCarParts NClient's for the Macintosh and the PC differ in their UI.

Issues introduced by code sharing include:

- **Single supplier/multiple consumers.** Core subsystems are used by teams working on different variants, and, thus, with possibly diverging requirements. The core subsystem teams need to satisfy their requirements uniformly.
- **Long change request turnaround.** When one variant-specific team issues a change request for a core subsystem, the approval and implementation of the change can take longer than for other changes, given that the core subsystem team needs to ensure the change does not interfere with the other variant-specific teams.
- **Cross platform inconsistencies.** Core subsystems introduce constraints on variant specific subsystems that can interfere with platform constraints. For example, a core subsystem can be designed with a threaded control flow in mind, whereas the user interface toolkit of a specific variant assumes an event-driven flow of control.

Each of these issues can be perceived by variant-specific teams as a motivation to implement their own core subsystems. These issues can be addressed, however, by anticipating variant-specific issues during system design and through effective configuration management. A good system design is resilient to platform and variant-specific issues. This results in a subsystem decomposition that is identical for all variants, where each system variant differs by substituting one or more subsystems. For example in Figure 10-13, the Macintosh and PC variants of the NClient share the Parts Catalog and the Server Interfaces subsystem. The user interface and the network interface subsystems are different but have the same interface. This results in a subsystem decomposition where each subsystem is either variant independent (i.e., supporting all variants) or variant specific (i.e., supporting a small number of variants). The issues we raised before can then be addressed as follows.

- The *single supplier/multiple consumers* issue is addressed by careful change management: If a requested change is variant specific, it should not be addressed in a core subsystem. If the requested change benefits all variants, then it should be addressed only in the core subsystems.
- The *long change request turnaround* is shortened by involving the team that issued the change request during validation. The suggested change is implemented in a new promotion of the core subsystem and released to the team who requested the change. The team evaluates the solution and tests its implementation, whereas other teams continue using the former promotion. Once the change is validated, other variant teams may start using the revised subsystem. We described such a scenario with the Serverside changes required by the navigation map (see Figure 10-9): The novice client team validated the new version of the server, while the EClient team continued working with a former (and more stable) release. Note that when variant teams require concurrent changes to the core subsystem, core developers can use branches to isolate the impact of each change until they reach a stable state.
- *Cross platform inconsistencies* are as much as possible avoided during system design by focusing on a variant independent subsystem decomposition. Lower level cross-platform inconsistencies are addressed in variant-specific subsystems, at the cost of some glue objects or redundancy between core and variant-specific subsystems. If all else fails, independent development paths should be considered when the supported variants are substantially different.

Managing multiple variants with shared subsystems is complex, as we indicated earlier. After an upfront investment during system design, however, the shared code approach yields numerous advantages, such as increased quality and stability of the shared code, and greater consistency in quality across variants. Finally, when the number of variants is large, considering variant-specific issues early and designing configuration management mechanisms to meet them lead to substantial savings in time and cost.

10.4.6 Change management

The creation of new promotions and releases is driven by change requests. A team repairs a fault and creates a promotion for quality control to evaluate. Clients define new requirements or developers take advantage of a new implementation technology causing a new release of the system. Change requests can range from correcting a typo in a menu label to reimplementing a major subsystem to address performance issues. Change requests also vary in their timing: A request for additional functionality during the requirements review can lead to modifying the draft RAD, whereas the same request made during testing can additionally lead to major subsystem surgery. Change requests need to be handled differently depending on their scope and their timing. The handling of change requests, called change management, is part of configuration management.

Change management processes vary in their formality and complexity with the project goals. In the case of a complex system with high reliability requirements, for example, the change request form has several pages (e.g., [MIL Std. 480]), requires the approval of several managers, and takes several weeks to be processed. In the case of a simple tool written by one developer, informal communication is sufficient. In both cases, the change process includes the following steps (Figure 10-14).

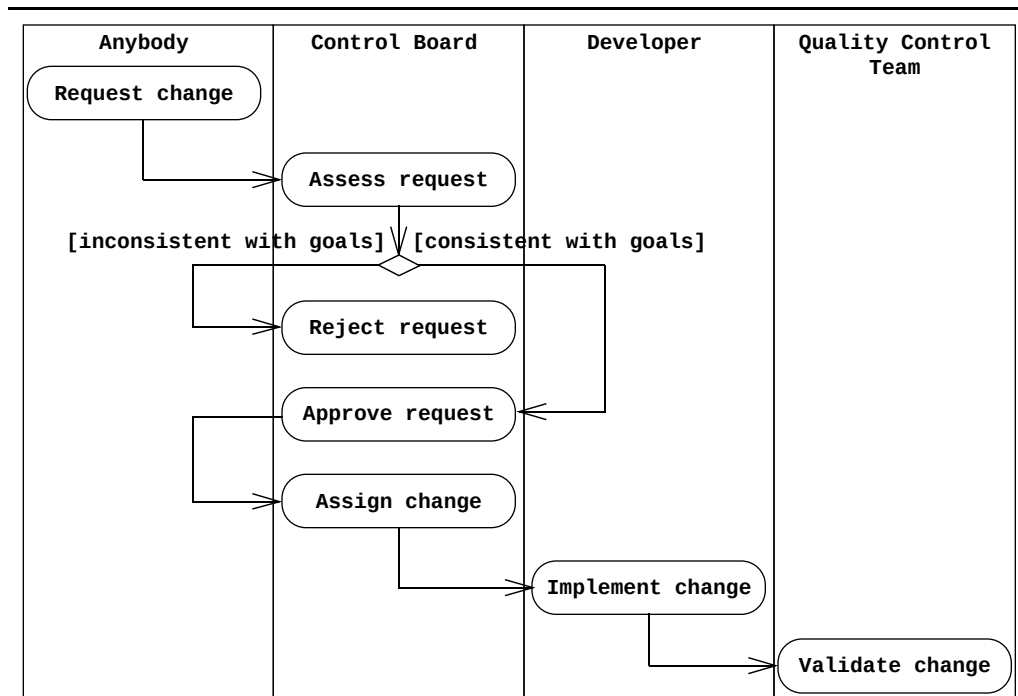


Figure 10-14 An example of change management process (UML activity diagram).

1. The change, identifying a fault or a new feature, is requested. This can be done by any one, including a user or a developer.
2. The request is assessed against project goals. In large projects, this is done by a control board. In smaller projects, this is done by the project manager. This may include a cost benefit analysis and an evaluation of impact of the change on the rest of the system.
3. Following the assessment, the request is either accepted or rejected.
4. If it is accepted, the change is planned, prioritized, and assigned to a developer and implemented.
5. The implemented change is audited. This is done by the quality control team or whoever is responsible for managing releases.

10.5 Managing configuration management

In this section, we address management issues related to configuration management. These include:

- documenting configuration management
- assigning configuration management responsibilities
- planning configuration management activities

10.5.1 Documenting configuration management

[IEEE Std. 828-1990] is a standard for writing *Software Configuration Management Plans* (SCMP). Such a plan documents all the relevant information to the configuration management activities of a specific project. The plan is generated during the planning phase, put under configuration management, and revised as necessary. The scope and the length of the plan may vary according to the needs of the project. A mission critical project with a formal change control process may require a 30-page document. A project developing a conceptual prototype may only require a five-page document.

An SCMP contains six types of information: introduction, management, activities, schedules, resources, and plan maintenance.

The *Introduction* describes the scope and audience of the document, key terms, and references.

The *Management* section describes the organization of the project (which can be a reference to the corresponding section in the *Software Project Management Plan*, see Chapter 11, *Project Management*) and how the configuration management responsibilities are assigned within this organization.

The *Activities* section describes in detail the identification of configuration items, the change control process, the process for creating releases and for auditing, and the process for status accounting. Responsibilities for each of these activities are assigned to a role defined in the management section.

Software Configuration Management Plan

1. Introduction
 - 1.1 Purpose
 - 1.2 Scope
 - 1.3 Key terms
 - 1.4 References
 2. Management
 - 2.1 Organization
 - 2.2 Responsibilities
 3. Activities
 4. Schedule
 5. Resources
 6. Plan Maintenance
-

Figure 10-15 An example of a template for the *Software Configuration Management Plan*.

The *Schedule* section describes when the configuration management activities take place and how they are coordinated. In particular, it defines at which point changes can be requested and approved through the formal change process.

The *Resources* section identifies the tools, techniques, equipment, personnel, and training necessary for accomplishing the software configuration activities.

Finally, the *Plan Maintenance* section defines how the SCMP, itself under configuration management, is maintained and revised. In particular, this section details the person responsible for its maintenance, the frequency of updates, and the change process for updating the plan.

The [IEEE Std. 828-1990] standard is defined such that the above outline can be applied to any project, from a reliable systems design to a freeware product.

10.5.2 Assigning configuration management responsibilities

Configuration management is a project function that involves many different tasks and participants in the project. As in the case of system design, tasks that require consistency should be performed by a small number of persons. During system design, a small team of architects make decisions about the subsystem decomposition. Similarly, a small team of configuration managers identify the configuration items and CM aggregates that need to be controlled. As in the case of testing, quality control tasks such as release management should be performed by different participants than development tasks, such as object design and implementation.

Configuration management include the following roles.

- **Configuration manager.** This is the role responsible for identifying configuration items and CM aggregates. This role may also be responsible in defining the procedures for creating promotions and releases. This role is often combined with the role of architect.

- **Change control board member.** This role is responsible for approving or rejecting change requests based on project goals. Depending on the complexity of the change process, this role can also be involved in assessing the change and in planning accepted changes. This role is often combined with the role of team or project manager.
- **Developer.** This role creates promotions triggered by change requests or the normal activities of development. The main configuration management activity of developers is to check in changes and resolve merge conflicts.
- **Auditor.** This role is responsible for the selection and evaluation of promotions for release. The auditor is also responsible for ensuring the consistency and completeness of the release. This role is often accomplished by the quality control team.

The roles for configuration management are defined during project planning and assigned early, in order to ensure consistency. Reassigning configuration management roles too often can severely limit the benefits of configuration management and fail to control change.

10.5.3 Planning configuration management activities

Project managers should plan configuration management activities during the project initiation phase. Most of the configuration management procedures can be defined before the project starts given that they do not depend on the system itself, but rather, on project goals and nonfunctional requirements such as safety and reliability.

The key elements in planning configuration management are:

- definition of the configuration management processes
- definition and assignment of roles
- definition of change criteria, that is, what attributes of the work products can be changed and how late in the process
- definition of release criteria, that is, which criteria should be evaluated when auditing promotions for a release

Moreover, configuration management procedures and tools need to be in place before changes start taking place, so that they can be recorded, approved, and tracked.

10.6 Exercises

1. RCS adopts a reverse delta approach for storing multiple versions of a file. For example, assume a file has three revisions, 1.1, 1.2, and 1.3, RCS stores the file as of version 1.3, then, the differences between 1.2 and 1.3, and the differences between 1.1 and 1.2. When a new version is created, say 1.4, the difference between 1.3 and 1.4 is computed and stored, and the 1.3 version is deleted and replaced by 1.4.

Explain why RCS does not simply store the initial version (in this case 1.1) and the differences between each successive version.

2. CVS uses a simple text-based rule to identify overlaps during a merge: There is an overlap if the same line was changed in both versions that are being merged. If no such line exists, then CVS decides there is no conflict and the versions are merged automatically. For example, assume a file contains a class with three methods, `a()`, `b()`, and `c()`. Two developers work independently on the file. If they both modify the same lines of code, say the first line of method `a()`, then CVS decides there is a conflict.

Explain why this approach will fail to detect certain types of conflict. Provide an example in your answer.

3. Configuration management systems such as RCS, CVS, and Perforce use file names and their paths to identify configuration items. Explain why this feature prevents the configuration management of CM aggregates, even in the presence of labels.
4. Explain how configuration management can be beneficial to developers, even in the absence of a change control or auditing process. List two scenarios illustrating your explanation.
5. In Chapter 8, *Rationale Management*, we described how rationale information can be represented using an issue model. Draw a UML class diagram for a problem tracking system that uses an issue model for the description and discussion of changes and their relationship with versions. Focus only on the domain objects of the system.
6. In Chapter 9, *Testing*, we described how the quality control team find faults in promotions created by subsystem teams. Draw a UML activity diagram including the change process activities and testing activities of a multiteam project.

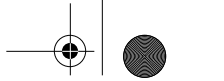
References

- | | |
|------------------------|--|
| [Babich, 1986] | W. A. Babich, <i>Software Configuration Management</i> . Addison-Wesley, Reading, MA, 1986. |
| [Berliner, 1990] | B. Berliner, "CVS II: parallelizing software development," <i>Proceedings of the 1990 USENIX Conference</i> , Washington, DC, pp. 22–26, Jan. 1990. |
| [Bersoff et al., 1980] | E. H. Bersoff, V. D. Henderson, & S.G. Siegel, <i>Software Configuration Management: An Investment in Product Integrity</i> . Prentice Hall, Englewood Cliffs, 1980. |
| [Conradi et al., 1998] | R. Conradi & B. Westfechtel, "Version models for software configuration management," <i>ACM Computing Surveys</i> , vol. 30, no. 2, June 1998. |
| [Dart, 1991] | S. Dart, "Concepts in configuration management systems," <i>Third International Software Configuration Management Workshop</i> , June 1991. |
| [IEEE Std. 828-1990] | <i>IEEE Standard for Software Configuration Management Plans</i> , IEEE Standards Board, Sept. 1990, in [IEEE 1997]. |
| [IEEE Std. 1042-1987] | <i>IEEE Guide to Software Configuration Management</i> , IEEE Standards Board, Sept. 1987 (Reaffirmed, Dec. 1993), in [IEEE 1997]. |
| [IEEE, 1997] | <i>IEEE Standards Collection Software Engineering</i> , Piscataway, NJ, 1997. |
| [Kemerer, 1997] | C. F. Kemerer, "Case 7: Microsoft Corporation: Office Business Unit," in <i>Software Project Management: Readings and Cases</i> . Irwin/McGraw-Hill, Boston, MA, 1997. |
| [Knuth, 1986] | D. E. Knuth, <i>The TeXbook</i> . Addison-Wesley, Reading, MA, 1986. |

Exercises

405

- [Leblang, 1994] D. Leblang, "The CM challenge: Configuration management that works," in *Configuration Management*, W. F. Tichy (ed.), vol. 2 of *Trends in Software*, Wiley, New York, 1994.
- [MIL Std. 480] MIL Std. 480, U.S. Department of Defense, Washington, DC.
- [Perforce] Perforce, Inc., 2420 Santa Clara Ave., Alameda, CA.
- [POSIX, 1990] *Portable Operating System Interface for Computing Environments* IEEE Std. 1003.1, 1990.
- [Raymond, 1998] E. Raymond, "The cathedral and the bazaar," Available at <http://www.tuxedo.org/~esr/writings/cathedral-bazaar/cathedral-bazaar.html>, 1998.
- [Steelman, 1978] *Requirements for high order computer programming languages: Steelman*, U.S. Department of Defense, Washington, DC, 1978.
- [Tichy, 1985] W. Tichy, "RCS - a system for version control," *Software Practice and Experience*. vol. 15, no. 7, 1985.



11

11.1	Introduction: The STS-51L launch decision	408
11.2	An overview of project management	409
11.3	Management concepts	413
11.3.1	Teams	414
11.3.2	Roles	419
11.3.3	Work products	422
11.3.4	Tasks	423
11.3.5	Schedule	425
11.4	Project management activities	427
11.4.1	Intelligent office buildings	428
11.4.2	Initiating the project	429
11.4.3	Project monitoring	441
11.4.4	Risk management	443
11.4.5	Project agreement	447
11.4.6	Client acceptance test	448
11.4.7	Installation	448
11.4.8	Postmortem	448
11.5	Managing project management models and activities	449
11.5.1	Documenting project management	449
11.5.2	Assigning responsibilities	451
11.5.3	Communicating about project management	452
11.6	Exercises	453
	References	454

